Using Lockahead to Improve Parallel Write Performance during RAID Integrity Checks

Oakley Brunt, Met Office

Introduction

While testing applications developed using the Met Office's LFRic library, Scientists have reported increased model runtimes coinciding with integrity checks of the Redundant Array of Independent Disks (RAID) system. The system is checked on the first day of every month at 0100Z and can last several days. If the reports of increased model runtimes are correct, this could impact development and, potentially, operations if not investigated.

The Met Office HPCs are configured with GridRAID – 41 drive arrays each containing two distributed spare volumes and RAID 6 stripes configured with 8 data units and 2 parity units per stripe (HPE, 2019). RAID 6 offers greater security in the event of disk failure through double parity (compared to RAID 5 which has single parity). However, the added security of RAID 6 comes with the price of slower writes due to additional parity calculations after each write (compared to RAID 5).

Slow write speed cannot be attributed entirely to the parallel file system. Configuration of the IO server, their use of APIs to libraries such as HDF5 and NetCDF, and these libraries' use of MPI-IO are all potential causes of non-optimal write speeds. In essence, it is important to consider the entire parallel IO stack when discussing IO.

To improve the performance of parallel writes without modifying the IO server or middleware, the MPI-IO system can be configured through environment variables. Collective buffering has been noted as particularly beneficial to parallel IO when configured optimally (Moore, 2018). By gathering and reorganising data before performing a (larger) write to disk, collective buffering minimises the number of small IO operations that are performed. This reduces the number of parity calculations that are performed for RAID 6-based filesystems. To configure this for parallel IO, the number of collective buffering nodes should be set equal to the number of writer nodes via MPI-IO hints.

Ensuring contention-free writes to a file requires file locking. The default lock configuration on HPE systems give locks to each MPI rank accessing the file. Locks are distributed to MPI ranks based on the current locks in the file and, if no locks are present, a lock on the entire file is provided (Moore, 2018). When that lock is relinquished and the next writer rank requests a lock, it may be granted another full-file lock. This method can cause a bottleneck on writing when many writer ranks are present.

For some cases, a more optimal lock mode is the Lockahead mode (Moore, 2018). In this mode, a set of non-overlapping locks are distributed to MPI ranks ahead of time, matching the file access patterns expected by the application. This avoids the problem of full-file locks, so file access should be much better with multiple writer ranks.

Every 1st of the month at 0100Z the RAID system is checked for data integrity, parity drive integrity and, in the event of degradation, triggers reconstruction of data. The process is slow, usually taking multiple days, and requires lots of read and, for reconstructions, write operations. Though the process is a background operation, system calls are required to access the data on OSTs, and these checks could be impacting LFRic model runtimes.

To test this, write operations during and outside a RAID check period can be measured and their data throughput compared.

Method

I. Data Collection

A continually submitting LFRic Atmosphere model was prepared that triggered a subsequent but independent model run on the successful completion of the current model.

A C224 resolution (301,056 cells in 2D) was chosen as a balanced coarseness that would allow running on multiple compute nodes while also using fewer temporal and physical resources compared to an operational-scale C896 resolution (~4.8m cells in 2D). The job requested 5 compute nodes for 588 ranks, resulting in a decomposition of 32 x 16 cells per partition.

LFRic relies on XIOS, the XML-configured IO Server, to write the model diagnostic files in

NetCDF format. The standard configuration for the C224 job is 64 XIOS ranks clustered on one node – a serial IO setup. To test parallel IO, 2 and 4 XIOS nodes were also tested.

To compliment the parallel XIOS write processes, the output directory for diagnostic files was tested with and without Lustre striping, see Table 1. All testing was performed on EXZ, the Met Office Development system, where the maximum number of flash-pool stripes available is 2.

XIOS nodes	Stripes
1	1
2	1
2	2
4	1
4	2

Table 1. Each row in the table represents a basic suite configuration.

To enable Lockahead mode, the Cray environment variable MPICH_MPIIO_HINTS was used. Specifically, the hints `cray_cb_write_lock_mode=2` and `cray_cb_nodes_multiplier=` (set equal to the number of XIOS nodes) were applied to the test group. The control group did not have these hints applied.

The full setting appeared as follows:

```
MPICH_MPIIO_HINTS=*:cray_cb_write_lock_mode=2:cray_cb_nodes_multiplier=2
```

To gather statistics on write rates to each file, two Cray MPICH environment variables were used: MPICH_MPIIO_STATS and MPICH_MPIIO_TIMERS. The output from these two environment variables allowed the collection of net write rate (GiBs/second) for each diagnostic file.

Data were collected outside of a RAID check, acting as the control data, and during a RAID check for the configurations listed in Table 1.

II. Data Analysis

For equality, a random sample of 3500 measurements was created from each configuration (XIOS nodes, striping, RAID, lock mode). These were used as the data for the analysis.

Preliminary analysis of violin and empirical CDF plots (see appendix) showed that net write rate measurements were strongly non-normally distributed, exhibiting substantial skew, heavy tails and, in some cases, bimodality (see figures). Because these violations of normality invalidate the assumptions of parametric tests, I used non-parametric, rank-based statistical methods.

I first tested the impact of RAID checks on write rates, separated by lock mode for fairness. To do this, I quantified the effect of RAID checks using Cliff's delta, a rank-based measure of effect size that measures the probability that a randomly selected observation from one condition exceeds a randomly selected observation from another. This method makes no distribution assumptions which makes it robust for the collected data (Meissel, 2024).

Formally, for measurements outside of a RAID check $\boldsymbol{X}$ and during a RAID check $\boldsymbol{Y}$,

$$\Delta = P(Y > X) - P(Y < X)$$

where positive values indicate a performance penalty associated with RAID checks.

To assess statistical significance, I used a permutation test. For each condition (Lockahead or standard), I pooled the write rate measurements from both RAID and control groups. I then randomly shuffled the RAID data labels across the pooled data and recomputed Cliff's delta for each condition. I calculated differences for 1000 permutations and compared these to the observed delta, giving me a p-value for each group as the proportion of permuted Cliff's delta statistics whose absolute value was at least as large as the observed value.

I then repeated these steps to measure the effect of lock mode, separating the data into control and RAID groups.

To determine whether Lockahead mitigates the performance impact of RAID checks, I used a non-parametric difference-in-differences (DiD) analysis. This approach explicitly tests whether performance differences during RAID checks differs depending on whether Lockahead mode is enabled.

For each locking configuration (Lockahead, Standard), I used Cliff's delta to get the observed effect of lock mode on write rates during and outside of a RAID check. I took the observed DiD

statistic as the difference between RAID and control delta values.

To assess statistical significance, I used the same permutation test. I shuffled the locking labels and recalculated the DiD statistic on each permutation. Finally, the p-value was given as the proportion of permuted DiD statistics whose absolute value was at least as large as the observed DiD statistic.

Results

I. Effects of RAID

RAID checks had a negative effect on net write rate in every category, though the effect size was negligible ($0.15 > \Delta > -0.15$) in nearly all cases (Meissel, 2024). The effect was small but not negligible for serial IO when using Lockahead mode ($\Delta = -0.25$, $p < 0.01$), see Figure 1.

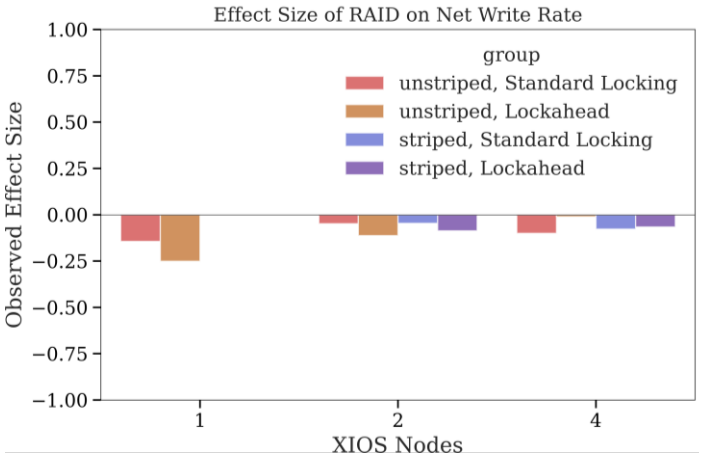


Figure 1. The effect size of RAID checks on net write rate (GiBs/s). Negative values indicate that RAID checks caused a decrease in net write rate, though values between -0.15 and 0.15 are negligible.

II. Effects of Lockahead

Enabling Lockahead had a medium ($-0.33 > \Delta > -0.47$) negative effect on net write rates for serial IO during both control ($\Delta = -0.44$, $p < 0.01$) and RAID ($\Delta = -0.46$, $p < 0.01$). In contrast, the effect sizes of Lockahead were large ($\Delta > 0.47$) and positive for all other test cases. The largest effect size was seen for 4 XIOS nodes with a single stripe during the control period ($\Delta = 0.75$, $p < 0.01$), as shown in Figure 2.

The percentage differences in median write rates were largest when using 4 XIOS nodes. The Lockahead rates were all >200% greater than their

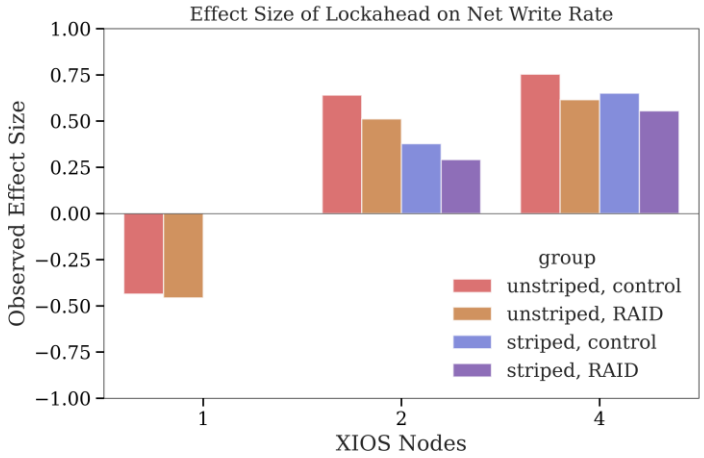


Figure 2. The effect size of Lockahead mode on net write rate (GiBs/s). Positive values indicate that Lockahead mode caused an increase in net write rate, and values greater than 0.47 are large.

Standard locking counterparts, while for 2 XIOS nodes, all differences fell between 130% and 160% greater. For 1 XIOS node, the Standard lock mode reported higher rates, with Lockahead rates ≤90% of their Standard counterparts (see Appendix for violin and eCDF plots). The largest median of measured net write rates was recorded during the control period using 4 XIOS nodes and 2 Lustre stripes, which reached 8.38 GiBs/second. The sample containing the highest individual net write rate was during the RAID check using 4 XIOS nodes and 2 Lustre stripes, recorded at 8.44 GiBs/second. All raw and analysed data are available on GitHub (see Appendix).

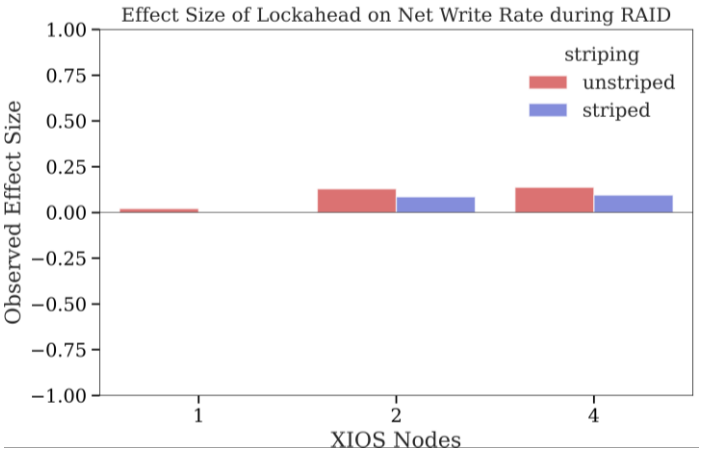


Figure 3. The effect sizes of Lockahead mode between control and RAID periods on net write rate (GiBs/s) compared to the effect sizes of standard locking mode under the same conditions, grouped by number of Lustre stripes (1 or 2). Values between -0.15 and 0.15 are negligible.

III. Effects of Lockahead during RAID

In all cases, the ability of Lockahead mode to reduce the impact from RAID checks on net write

rate was negligible. The largest effect was seen in the configuration using 2 XIOS nodes and 1 Lustre stripe ($\Delta = 0.13$, $p < 0.01$), however this is still a negligible effect. For serial IO (1 XIOS node, 1 Lustre stripe), the effect was negative, though the result was not statistically significant ($\Delta = -0.02$, $p = 0.235$) and the effect size was negligible. The results are drawn in Figure 3.

Discussion of Results

The impacts of RAID checks on write rates are mainly seen for serial IO, and with parallel IO there is barely a drop in rates. This is consistent with the motivation for this study; RAID checks may be causing slower model runtimes due to reduced data throughput. This seems plausible when looking at serial IO performance and knowledge that most development suites in LFRic Applications use serial IO due to their resolution (coarse for faster testing). This is likely where the anecdotal evidence originates, since the effects of RAID are not found to be significantly large when using parallel IO.

Lockahead was detrimental to serial IO jobs in the tested configuration. Enabling Lockahead, using a single writer, and setting the collective buffering multiplier to 1 is not optimal, as evidenced by Figure 2. This is likely attributable to a full-file lock being more efficient for serial IO since relinquishing and granting a file lock to the same process will incur overheads (Moore, 2023).

Lockahead seems very powerful with any amount of parallel IO. As stated, net write rates were over 2x larger than standard locking when using 4 XIOS nodes, irrespective of stripes or RAID. When using 2 XIOS nodes, the effect of Lockahead is more pronounced in unstriped configurations. This suggests that the impact of Lustre striping has some benefits in terms of write rate, though not as large as Lockahead.

From the DiD analysis, Lockahead showed very little impact on the change in write rate during RAID checks. The parallel IO effects did move from negative to positive, suggesting that Lockahead has *some* mitigating effects in this scenario. However, both the impact of RAID on write-rates and the DiD analysis found negligible effect sizes, suggesting that RAID checks are not a significantly detrimental factor for parallel writing, and that Lockahead does not perform better or worse than standard locking mode in mitigating the small impacts of RAID checks.

This study was limited by the number of Lustre stripes available on the Met Office Development system (2). The LFRic Atmosphere configuration was not equivalent to an operational model, and the volume of data was therefore smaller. To broaden the study, tests with a C896 configuration would allow more targeted recommendations to be made and further investigations with greater numbers of Lustre stripes would provide more certainty in the effects of Lockahead mode.

Conclusions

This research found that RAID integrity checks can reduce data throughput by a non-negligible amount when using serial IO. By contrast, parallel IO can successfully mitigate the impacts of RAID checks and does not require further tuning to achieve this result. This demonstrates that parallel IO is good practice, not just because it can achieve greater data throughput, but also that it can provide more stability for a given IO configuration.

Further tuning was tested here and showed that parallel IO can be optimised easily and effectively. In particular, combining Lockahead and Lustre striping as elements of a parallel IO configuration can achieve much greater write rates than Standard lock mode and striping.

Glossary

Collective Buffering

A parallel IO technique that collects IO requests across processes, reorganizes them via an aggregator node, and then performs a larger, more optimal write or read to disk. This can reduce time spent on IO operations by large amounts if configured optimally.

Parity

Single Parity (RAID 5) is the storage of disk data on more than one OST (striping) with an additional parity disk storing a checksum of the striped data. Each disk contains some of the parity data and therefore, in the event of a disk failure, the data can be rebuilt from the remaining disks.

Double Parity

Double Parity means that there are two parity drives. In the event of two disks failing the data can be rebuilt (this could be a parity disk and a data disk, or two data disks). This provides better security than RAID 5 which has single parity and would only be able to rebuild data from a single failed disk.

Data Throughput

The rate at which data travel across a network.

References

HPE (2019). ClusterStor 9000 Replacement Procedures 2.0.0 H-6191. Retrieved from https://support.hpe.com/hpesc/public/docDisplay?docId=a00113859en_us&page=GridRAID_Recovery.html

Meissel, K. and Yao, E.S., 2024. Using Cliff's delta as a non-parametric effect size measure: an accessible web app and R tutorial. *Practical Assessment, Research, and Evaluation*, *29*(1).

Moore, M., Farrell, P. and Cernohous, B., 2018. Lustre lockahead: Early experience and performance using optimized locking. *Concurrency and Computation: Practice and Experience*, *30*(1), p.e4332.

Moore, M., Reghunandanan, A. and Gerhardt, L., 2023. MPI-IO Local Aggregation As Collective Buffering for NVMe Lustre Storage Targets.

Appendix

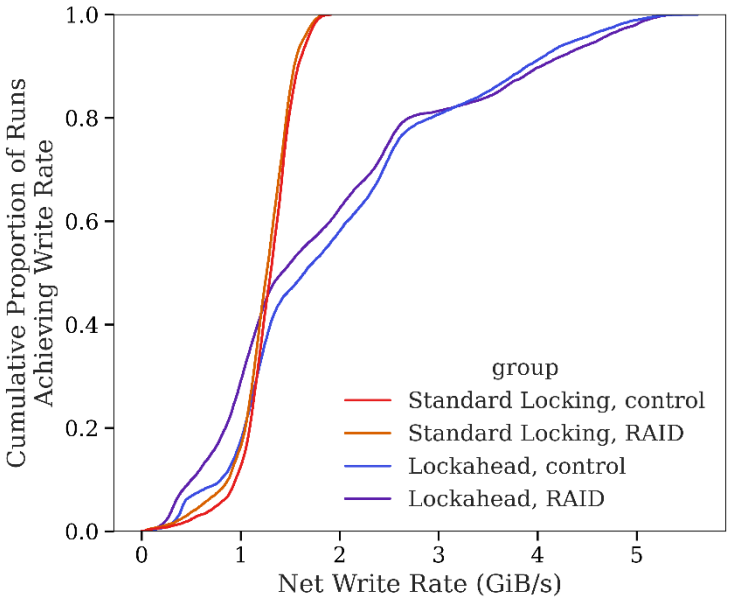
GitHub Repository

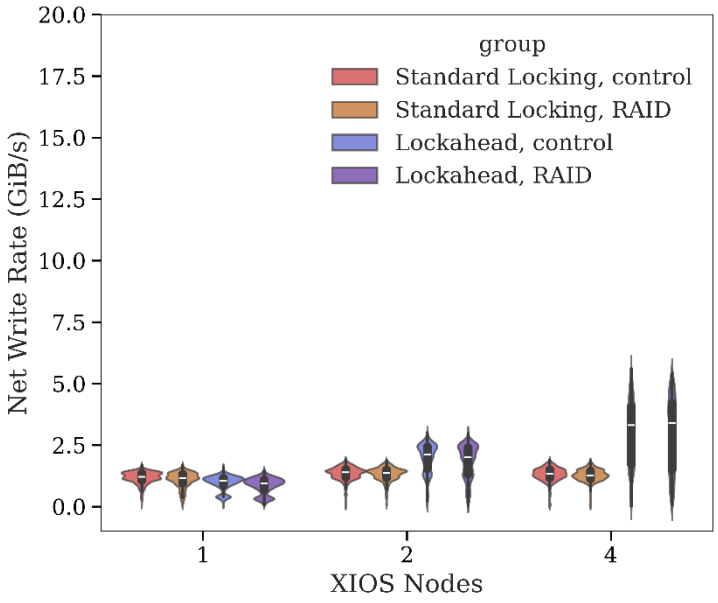
All collected and analysed data, along with functions used for analysis and plots can be found on GitHub at https://github.com/oakleybrunt/RAID-check-study

Violin and Empirical CDF plots

See next page.

Unstriped output





Striped output